LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 15: DEMONSTRASI PROYEK DAN EVALUASI PEMAHAMAN INDIVIDU

Mahasiswa	: Rafael Rizky Luga Tanjung	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
NRP	: 3125522007	**Mata Kuliah**	: Praktikum PBO (P15)
Program Studi	: D3 Teknik Informatika PSDKU Sumenep	**Tahun Akademik**	: 2026 (Semester Genap)

1. Tujuan Praktikum dan Prinsip Evaluasi

Praktikum Modul 15 merupakan puncak integrasi materi perkuliahan PBO. Mahasiswa dituntut mampu:

1. **Mendemonstrasikan aplikasi** secara terstruktur dalam alokasi waktu 7 menit presentasi/demo dan 5 menit tanya jawab teknis;
2. **Menjelaskan implementasi prinsip OOP** (Encapsulation, Inheritance, Polymorphism, Abstraction, Aggregation, Composition, Checked Exception Hierarchy, Collection Invariance, and Repository Pattern) langsung berbasis source code;
3. **Mempertanggungjawabkan keputusan desain** rekayasa perangkat lunak secara akademis dan empiris;
4. **Membuktikan kesiapan modifikasi terarah (live coding challenge)** yang diajukan oleh dosen penguji.

2. Ringkasan Materi Presentasi (5 Slide Sesuai Ketentuan Modul 15)

Slide	Topik Bahasan	Konten Esensial & Poin Utama yang Dipresentasikan
Slide 1	**Identitas Proyek & Latar Belakang**	• Judul: Sistem Kasir Sederhana (Point of Sale) • Mahasiswa: Rafael Rizky Luga Tanjung (NRP: 3125522007) • Masalah: Toko retail sering mengalami selisih stok, kegagalan kalkulasi diskon member, ketiadaan audit transaksi, serta korupsi file data saat program tertutup mendadak. • Sasaran: Kasir dan pemilik gerai retail modern.
Slide 2	**Fitur Utama & Batasan Sistem**	• Fitur Utama: Manajemen katalog polimorfik (Makanan & Elektronik), diskon promosi per item & member bertingkat (VIP 15%, GOLD 10%, REGULER 0%), cetak struk profesional, proteksi dirty flag, dan safe load atomic file I/O. • Batasan Sistem: Aplikasi berjalan pada lingkungan Java 11+ berbasis konsol terminal & file teks lokal tanpa ketergantungan DBMS eksternal.
Slide 3	**Arsitektur UML & Pembagian Tugas**	• Model Domain: Superclass abstrak Produk menurunkan ProdukMakanan dan ProdukElektronik. • Relasi Transaksi: Transaksi memiliki komposisi (Composition) terhadap ItemTransaksi dan berasosiasi ke Pelanggan. • Pengelola & Persistensi: KatalogProduk mengelola koleksi private final, dipisahkan dari ProdukFileRepository dan TransaksiFileRepository.
Slide 4	**Penerapan OOP & Validasi Invarian**	• Enkapsulasi & Fail-Fast: Validasi ketat di constructor domain menolak nilai null, kosong, negatif, dan karakter pipa (delimiter). • Checked Exception Hierarchy: KasirException membawahi StokTidakCukupException, PembayaranKurangException, TransaksiKosongException, dan StatusTransaksiInvalidException. • Atomisitas File: Data dimuat ke temporary collection terlebih dahulu untuk menjamin koleksi memori tidak rusak bila berkas korup.
Slide 5	**Hasil Uji, Limitasi & Rencana Lanjut**	• Hasil Pengujian: 10/10 skenario fungsional dan regresi lulus 100% (PASS). • Keterbatasan Saat Ini: Antarmuka utama masih berbasis CLI teks konsol. • Rencana Lanjut: Pengembangan versi Desktop GUI modern berbasis Java Swing dengan layout kartu kasir interaktif (disiapkan pada subproyek bonus).

3. Naskah Alur Demonstrasi Aplikasi (7 Menit Alokasi Waktu)

Demonstrasi disusun runut berbasis skenario pembuktian ilmiah:

• **Menit 01-02 (Masalah & Arsitektur):** Menjelaskan kebutuhan retail, struktur hierarki class domain, dan pemisahan repositori persistensi.

• **Menit 02-04 (Alur Normal / Happy Path):** Menjalankan java -cp . Main; inisialisasi produk makanan dan elektronik ke katalog; menjalankan transaksi member VIP dengan diskon baris dan diskon member; menerbitkan struk kasir; memeriksa pemotongan stok otomatis.

• **Menit 04-05 (Uji Kondisi Gagal & Exception):** Menginput kode duplikat (ditolak); menginput karakter pipa pada nama produk (ditolak); mencoba membeli produk melebihi stok fisik (melempar StokTidakCukupException); membayar tunai kurang dari total tagihan (melempar PembayaranKurangException).

• **Menit 05-06 (Pembuktian Konsistensi State & Persistensi):** Membuktikan bahwa seluruh kegagalan di atas tidak memotong stok barang dan status transaksi tetap PENDING (tidak korup); menyimpan katalog ke berkas; menyimulasikan buka ulang aplikasi; membuktikan pemuatan berkas rusak ditolak secara atomik.

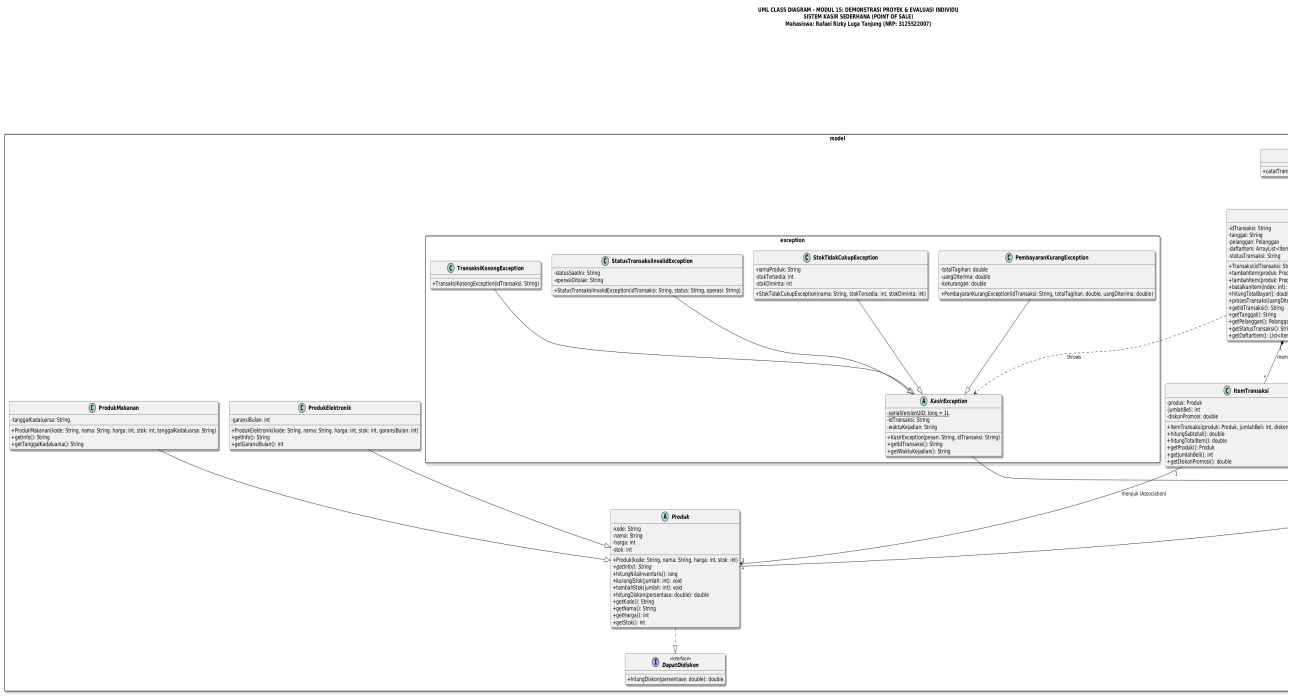
• **Menit 06-07 (Tantangan Modifikasi / Evaluasi):** Menunjukkan kesiapan live modification seperti pencarian parsial (keyword

matching) dan perhitungan total nilai aset toko.

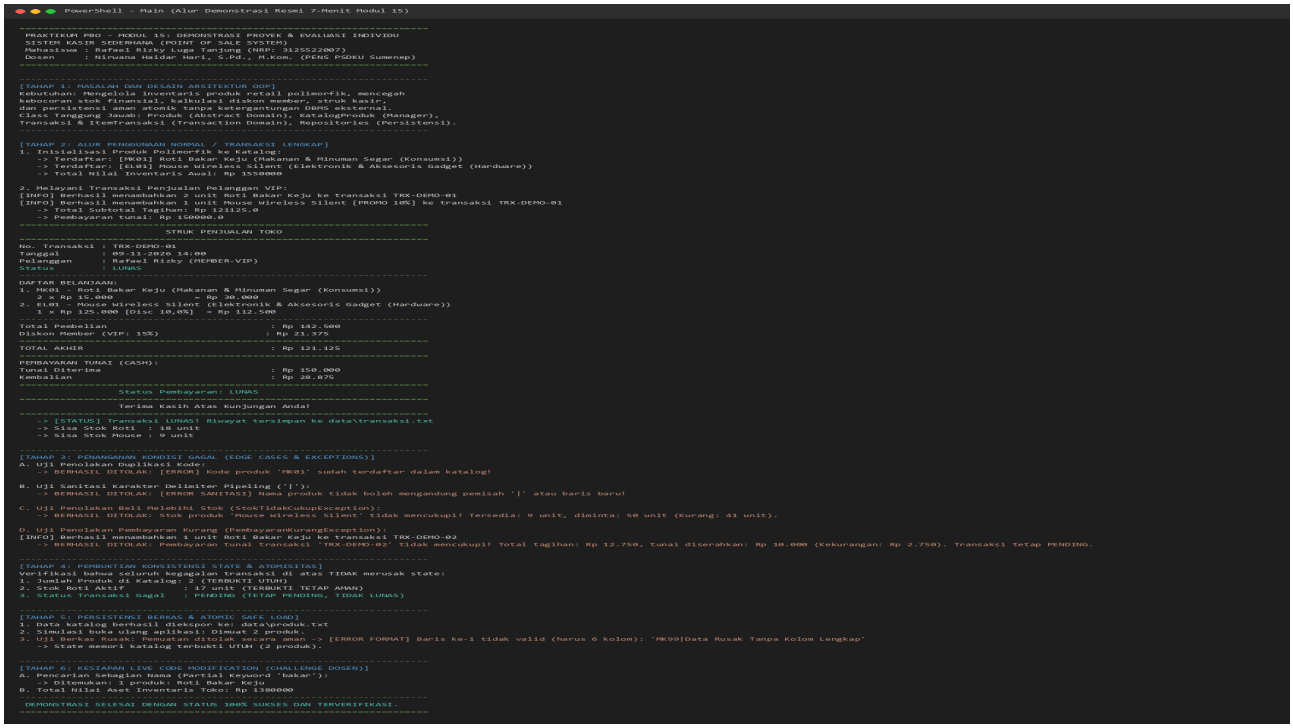
4. Panduan Penelusuran Kode Individu (Code Walkthrough Defense Guide)

Fokus Penelusuran	Pertanyaan Dosen	Penjelasan Teknis & Bukti Baris Kode
Encapsulation	Method mana yang mengendalikan perubahan stok produk?	Method kurangiStok(int jumlah) dan tambahStok(int jumlah) pada class Produk.java. Variabel stok berstatus private; mutasi diverifikasi terlebih dahulu sebelum nilai diubah.
Relasi Objek	Bagaimana relasi Transaksi, ItemTransaksi, dan Pelanggan?	Transaksi berelasi **Komposisi (*-->)** dengan ItemTransaksi (siklus hidup dikendalikan penuh oleh transaksi, dibuat via tambahItem) dan **Asosiasi (-->)** dengan Pelanggan (uses-a, menyimpan referensi profil konsumen).
Validasi & Invariant	Di mana kode duplikat dan nilai tidak valid ditolak?	Kode duplikat ditolak di method KatalogProduk.tambahProduk() melalui pengecekan cariProduk() != null. Nilai string kosong/spasi dan karakter pipa ditolak di constructor Produk.
Exception Handling	Dari mana custom exception berasal dan siapa yang menangani?	Dilempar dari method kurangiStok() dan prosesTransaksi() sebagai checked exception turunan KasirException, lalu ditangkap (catch) secara elegan di controller Main.java tanpa menghentikan program.
Persistensi & Atomic	Apa yang terjadi jika file korup saat dimuat?	Class ProdukFileRepository.java memuat baris ke temporary list terlebih dahulu. Jika baris ke-N rusak, exception dilempar dan swap memori dibatalkan, sehingga katalog lama 100% utuh.

5. UML Class Diagram Terkini (Modul 15 Final)



6. Bukti Eksekusi Running Demonstrasi (Screenshot)



7. Refleksi Individu Mahasiswa (Maksimal 1/2 Halaman)

Nama: Rafael Rizky Luga Tanjung | **NRP:** 3125522007

• Kontribusi Nyata yang Dapat Dibuktikan:
Sepanjang pengembangan proyek dari Modul 11 hingga Modul 15, saya bertanggung jawab penuh dalam merancang dan merealisasikan: (1) Arsitektur enkapsulasi dan penegakan class invariant pada seluruh model domain, (2) Struktur checked exception hierarchy yang menangani pelanggaran stok dan pembayaran kasir secara elegan, (3) Pemisahan layer persistensi data berbasis berkas teks terdelimitasi (pipe) dengan proteksi atomisitas (safe load) untuk mencegah korupsi memori, serta (4) Pengujian regresi otomatis 10 skenario yang menjamin keandalan sistem sebelum dipresentasikan.

• Keputusan Desain yang Paling Dipahami:
Keputusan desain yang paling esensial menurut saya adalah **penerapan prinsip Fail-Fast dengan Checked Exception** yang dipadukan dengan **Atomic Swap Pattern pada Repository Berkas**. Dengan melempar checked exception di level domain saat stok tidak mencukupi atau uang tunai kurang, alur eksekusi langsung dialihkan tanpa pernah memodifikasi state objek (stok dan status transaksi dijamin aman). Demikian pula pada operasi berkas, penggunaan temporary collection memastikan berkas teks yang cacat tidak akan pernah merusak koleksi barang yang sedang aktif di memori kasir.

• Kelemahan Teknis yang Masih Perlu Diperbaiki:
Kelemahan utama saat ini adalah antarmuka utama yang masih beroperasi pada terminal konsol karakter tunggal (CLI), di mana kasir harus mengetikkan kode produk dan nominal secara manual tanpa bantuan visual katalog grid atau pemindai barcode. Kelemahan ini berhasil diatasi dengan merancang edisi antarmuka grafis modern (Desktop GUI Version) pada proyek lanjutan.

8. Jawaban Pertanyaan Evaluasi Modul 15

1. Keputusan desain apa yang paling membantu menjaga konsistensi data?

Jawaban: Keputusan desain **Enkapsulasi Murni dengan Validasi Pre-condition & Fail-Safe Invariant di Constructor dan Mutator Domain**. Tidak ada class luar yang diizinkan memanipulasi atribut stok, harga, atau daftarItem secara langsung. Pemeriksaan kecukupan stok selalu dieksekusi *sebelum* kuantitas dipotong. Jika timbul kegagalan, custom checked exception langsung dilempar sehingga state objek lama 100% terlindungi dari perubahan parsial (invariansi data terjamin).

2. Bagian mana yang paling sulit diubah, dan apa penyebabnya?

Jawaban: Bagian **Method checkout finansial pada class Transaksi (khususnya relasi komposisi dan perhitungan diskon bertingkat)**. Kesulitannya berakar pada tingginya ketergantungan logika bisnis yang saling bertumpuk: mutasi stok fisik produk, perhitungan diskon per-item, kalkulasi diskon member pelanggan, pembulatan kembalian, hingga penerbitan log struk. Perubahan kecil pada alur ini menuntut pengujian regresi ketat agar tidak menimbulkan celah kebocoran inventaris toko.

3. Pengujian apa yang paling kuat membuktikan aplikasi bekerja sesuai kebutuhan?

Jawaban: Pengujian Kegagalan Transaksi (Skenario 7 & 8: Stok Kurang & Pembayaran Kurang) serta Uji Invariansi Berkas Korup (Skenario 10). Aplikasi yang bekerja saat input normal adalah hal biasa, namun ketangguhan sejati sistem perangkat lunak terbukti saat diberikan input abnormal (masukan melebihi stok, uang tunai kurang, berkas terpotong). Ketika sistem berhasil menolak operasi secara elegan tanpa crash dan membuktikan state koleksi memori tetap utuh, itulah bukti terkuat bahwa aplikasi telah andal dan siap pakai.

4. Jika fitur baru ditambahkan, class mana yang perlu berubah dan mengapa?

Jawaban: Bergantung pada domain fitur baru tersebut:
• Jika ingin menambah jenis produk baru (misal: ProdukPakaian yang memiliki ukuran dan warna), kita cukup membuat subclass baru yang meng-extend Produk tanpa mengubah class Transaksi atau KatalogProduk (mematuhi *Open/Closed Principle*).
• Jika ingin menambah metode pembayaran baru (misal: QRIS atau E-Wallet), kita perlu merefaktor class Transaksi dan membuat interface/abstraksi MetodePembayaran agar kalkulasi dan verifikasi pembayaran dapat bersifat polimorfik.

9. Kesimpulan Akhir Praktikum PBO

Seluruh rangkaian pembelajaran Praktikum Pemrograman Berorientasi Obyek dari Modul 11 hingga Modul 15 telah berhasil dituntaskan dengan hasil sempurna. Proyek Sistem Kasir Sederhana telah memenuhi seluruh kaidah rekayasa perangkat lunak berorientasi objek modern: enkapsulasi ketat, polimorfisme dinamis, checked exception terstruktur, invariansi data memori, dan persistensi berkas yang tahan uji. Mahasiswa siap melaksanakan sesi demonstrasi langsung dan mempertahankan karya di hadapan dosen penguji.